Placement Management System

DBMS Project Report

Project Title: Placement Management System
Technology Stack: HTML, CSS, JavaScript, Node.js, Express.js, MySQL, JWT, bcrypt, Chart.js
Prepared For: Final Year DBMS Project Submission
Prepared By: _____
Roll No.: _____
Department: _____
College: _____
Academic Year: 2025-26

Acknowledgement

I express my sincere gratitude to my project guide, faculty members, and the Department of Computer Science for their support and guidance during the development of this DBMS project. Their direction helped me understand how to design a structured database system, build REST APIs, and connect a browser-based frontend to a MySQL backend in a practical and reliable way.

I also acknowledge the use of open web technologies such as HTML, CSS, JavaScript, Node.js, Express.js, MySQL, JWT, bcrypt, and Chart.js, which made it possible to build a complete placement management application. Finally, I thank my classmates and family members for their encouragement throughout the project development process.

Table of Contents

SL.NO	TITLE	PAGE NUMBER
1	CONTENT'S ABSTRACT	4
2	INTRODUCTION	5
3	BACKGROUND, MOTIVATION AND SCOPE	6
4	METHODOLOGY	7
5	REQUIREMENTS	9
6	E-R DIAGRAM	10
7	RELATIONAL DATABASE DESIGN	12
8	DATABASE NORMALIZATION	14
9	DATA DICTIONARY	15
10	GRAPHICAL USER INTERFACE	17
11	SOURCE CODE	18
12	CONCLUSION	20
13	REFERENCES	21
14	BIBLIOGRAPHY	21

1. CONTENT'S ABSTRACT

The Placement Management System is a full-stack DBMS project developed to simplify and automate the campus placement process for colleges and universities. The system replaces manual tracking methods with a centralized web application where administrators can manage students, companies, job openings, applications, interviews, and placement statistics from a single interface. Students can log in securely, view available jobs, submit applications, and track interview-related updates without depending on paper records or repeated manual coordination.

The project uses a practical and modern technology stack. The frontend is built with HTML, CSS, and JavaScript, which makes the interface lightweight, responsive, and easy to use on standard browsers. The backend is implemented with Node.js and Express.js, exposing RESTful APIs for authentication, CRUD operations, and analytics. MySQL is used as the relational database to store structured placement data, while JWT provides token-based authentication and bcrypt is used to hash passwords securely. Chart.js powers the analytics dashboard, allowing the placement cell to visualize student strength, placement status, company-wise hiring, package distribution, application trends, and interview timelines.

The system is organized into modules that reflect the real placement workflow. The users module manages admin and student identities. The students module stores academic and contact information such as roll number, department, CGPA, skills, phone number, and resume link. The companies module records hiring organizations and their contact details. The jobs module stores job postings with eligibility criteria, package details, and deadlines. The applications module tracks which student applied to which job and what the current application status is. The interviews module schedules and tracks interview rounds, interview mode, venue, interviewer, result, and notes.

Database usage is a major strength of the project. The schema uses primary keys, foreign keys, unique constraints, and cascading deletes to maintain consistency across tables. The database currently ships with realistic seed data containing 21 users, 20 students, 8 companies, 15 jobs, 40 applications, and 29 interviews. This allows the project to demonstrate meaningful dashboard values and realistic placement scenarios during testing and viva demonstration.

The API layer is designed for clarity and maintainability. Authentication endpoints handle login and student registration. Module endpoints manage students, companies, jobs, applications, interviews, and dashboard analytics. The frontend communicates with these APIs using the browser fetch interface through a shared wrapper, so requests automatically include the JWT token when available. This creates a smooth request-response flow between browser, server, and database.

Another important feature of the project is the analytics dashboard. The admin dashboard shows total students, companies, jobs, applications, selected students, rejected students, placement percentage, highest package, average package, recent applications, upcoming interviews, and top offers. Chart.js visualizations provide quick insight into department distribution, placement status, company-wise hiring, applications over time, package bands, and interview scheduling patterns. This converts raw data into information that can be used for placement planning and decision-making.

The project outcome is a complete, beginner-friendly, and academically useful placement management platform that is suitable for final-year DBMS submission. It demonstrates database design, normalization, authentication, role-based access control, frontend-backend integration, and analytics reporting in a single working system. The project also leaves room for future growth, such as resume parsing, email notifications, company-side dashboards, and placement prediction features.

2. INTRODUCTION

A Placement Management System is a software application designed to manage the placement activities of a college or university in a structured and efficient manner. It stores information about students, companies, jobs, applications, and interviews in a centralized database and provides a browser-based interface for administrators and students. Instead of handling all placement work through spreadsheets, notice boards, and manual verification, the system automates repetitive tasks and keeps the placement process organized.

Automation is needed in placement management because placement cells deal with many records at the same time. A single academic year may involve hundreds or thousands of students, multiple recruiters, several job roles, changing eligibility criteria, interview schedules, and final selection outcomes. Manual handling makes it difficult to maintain accuracy, update records quickly, and generate reports on demand. Automation reduces human error, saves time, improves transparency, and makes it easier to track the status of every student and job opening.

The purpose of this project is to build a complete DBMS-based solution that supports the entire placement workflow. The objective is not only to store data, but also to make the data useful through login security, record management, application tracking, interview scheduling, and analytics. The system is designed so that an admin can manage placement operations from one dashboard while students can log in to see relevant jobs and their own placement progress.

The technologies used in this project were selected for their simplicity and practical value:

- **HTML** is used to create the structure of each page such as forms, tables, dashboards, and login screens.
- **CSS** is used to style the interface, provide layout, and support a clean dark mode and responsive design.
- **JavaScript** adds page interactivity, fetch-based communication with the backend, form behavior, theme switching, and chart rendering.
- **Node.js** runs the server-side JavaScript environment and makes it possible to build the backend in one language.
- **Express.js** provides a lightweight framework for REST APIs, middleware, routing, and static file serving.
- **MySQL** stores the relational data in normalized tables with keys and constraints.
- **JWT** is used for secure token-based authentication after login.
- **bcrypt** hashes passwords before storing them, improving security.
- **Chart.js** creates readable charts on the analytics dashboard.

Together, these technologies make the project practical for academic use and also realistic enough to explain during viva. The system demonstrates how a modern placement office can move from fragmented manual handling to a centralized database-driven platform.

Technology Role Summary

Technology	Role in the Project
HTML	Page structure and form layout
CSS	Styling, responsive layout, dark mode
JavaScript	Client-side logic and API calls
Node.js	Backend runtime
Express.js	REST API framework
MySQL	Structured data storage

Technology	Role in the Project
JWT	Authentication token mechanism
bcrypt	Password hashing
Chart.js	Analytics charts

3. BACKGROUND, MOTIVATION AND SCOPE

Traditional placement systems in colleges often rely on notices, Excel sheets, email chains, printed forms, and manual communication between the placement cell, students, and recruiters. Student eligibility is checked manually, job announcements are sent separately, interview schedules are shared through different channels, and final results are updated in disconnected records. This method works only when the data volume is small and the process is simple.

Manual placement handling creates several problems. Information can be duplicated across multiple files, updates may not reach all students at the same time, and the placement cell may lose time searching for records. Eligibility filtering becomes slow when the number of companies increases. Interview dates can overlap, and application statuses may not be updated consistently. In addition, manual methods do not provide immediate analytics such as the placement percentage, the number of selected students, or the department-wise performance summary that management needs for review.

The motivation behind developing this system came from the need for a more organized and reliable placement workflow. A single database can keep student profiles, company details, job posts, applications, and interviews in one place. A role-based web application can allow students to view opportunities while allowing only the admin to perform sensitive operations such as creating jobs, updating status, or scheduling interviews. This makes the workflow faster and easier to control.

The scope of the project in colleges and universities is broad. The system can be used by placement officers, department coordinators, and students during the campus recruitment season. It can support full-time placements, internships, and pre-placement offers. The database structure is suitable for institutions that want to maintain historical placement records across multiple years. Because the project uses a web browser interface, it can be accessed from lab systems, office desktops, or any device on the campus network.

The future scope of the project is also strong. Additional modules can be added for company-side login, automated resume parsing, email notifications, export to Excel or PDF, role-based access for faculty coordinators, and integration with an academic ERP. The system can also be expanded with placement prediction using student CGPA and skill data, interview feedback management, and advanced recruitment analytics. These enhancements would make the platform more useful in real institutional environments.

Key Problems Solved

- Centralized storage of placement data
- Faster student and job record management
- Secure login and access control
- Automatic relation between job applications and interviews
- Analytics for placement planning
- Better transparency for students and administrators

4. METHODOLOGY

The project was developed using a step-by-step web application methodology. The first step was to identify the placement workflow and divide it into logical modules: users, students, companies, jobs, applications, interviews, and analytics. After module identification, the database schema was designed in MySQL so that each entity could be stored in a separate table with suitable keys and relationships. Once the database structure was finalized, the backend APIs were implemented using Node.js and Express.js. The frontend pages were then developed using HTML, CSS, and JavaScript and connected to the APIs using fetch requests.

Frontend development focused on usability and clarity. The login page accepts email, password, and role. The admin dashboard displays KPIs, data tables, and charts. Separate pages exist for students, companies, jobs, applications, and interviews. The frontend JavaScript files are divided by module so that each page has its own script and logic. This keeps the code simple for beginners and makes debugging easier. The interface also supports dark mode, loading states, alerts, and responsive layouts.

Backend API development was done as a REST service. The server entry file, `backend/server.js`, loads environment variables, enables CORS, parses JSON bodies, serves the frontend as static files, and mounts the route modules. Each route file maps HTTP methods and URLs to controller functions. For example, `authRoutes.js` handles login and student registration, `studentRoutes.js` handles student CRUD, and `dashboardRoutes.js` provides statistics and analytics data. Validation middleware is used to catch invalid input before it reaches the database.

Database integration is handled through `mysql2/promise` using a connection pool in `backend/src/config/db.js`. A pool is a better choice than a single connection because it can manage multiple requests without creating a new connection every time. The controllers perform SQL queries using placeholders, which helps prevent SQL injection. Transactions are used when a user and student profile must be inserted together during registration or student creation.

Authentication flow is based on JWT and bcrypt. During login, the backend searches the `users` table using the provided email and role. It then compares the entered password with the stored bcrypt hash. If the credentials are valid, the server creates a JWT token containing the user ID, name, email, and role. The frontend stores the token and user details in localStorage. Protected pages call a helper that checks whether the token is present and whether the role is correct. Protected backend routes use `protect` and `allowRoles` middleware to verify the token and restrict access.

The analytics dashboard implementation is one of the strongest parts of the system. The backend calculates placement totals and returns chart-ready label-value arrays. The frontend loads these datasets and uses Chart.js to draw bar charts, doughnut charts, and line charts. The dashboard shows students by department, placement status, company-wise hiring, applications over time, package distribution, and interview timeline. This is helpful for the placement cell because it turns database records into visual insights.

Dark mode implementation is handled in the shared `ui.js` script. The selected theme is stored in localStorage under a theme key. When the page loads, the script reads the saved value and applies a CSS class to the body. The theme toggle button changes the value and dispatches a custom event so charts and UI elements can re-render using the active colors. This approach is simple, reusable, and works across all pages.

Project Architecture

The architecture follows a clean browser-client-server-database flow:

```
Browser Pages
  -> fetch() requests in frontend/assets/js
  -> Express.js routes and middleware
  -> Controller logic
  -> MySQL database pool
  -> JSON response back to the browser
```

At a functional level, the system contains four layers:

1. Presentation layer: HTML pages, CSS styling, charts, tables, forms.
2. Application layer: Node.js and Express.js route handling.
3. Business layer: controller logic for CRUD, login, analytics, and validation.
4. Data layer: MySQL tables and relational constraints.

Request-Response Flow

1. The user opens a page such as login, jobs, or dashboard.
2. JavaScript reads form input or triggers an API call.
3. The shared `api.js` wrapper sends a `fetch()` request.
4. If a token exists, the wrapper adds `Authorization: Bearer <token>`.
5. Express routes receive the request and apply middleware.
6. Controllers run SQL queries on MySQL.
7. The server returns JSON data or an error message.
8. The frontend updates the UI, tables, notifications, or charts.

Development Steps

1. Defined the placement workflow and role structure.
2. Designed the relational schema in MySQL.
3. Seeded the database with realistic sample records.
4. Built the Express backend and connection pool.
5. Added authentication with JWT and bcrypt.
6. Created CRUD APIs for each placement module.
7. Built the frontend pages and shared helpers.
8. Added dashboard analytics and Chart.js graphs.
9. Implemented dark mode and persistent local settings.
10. Tested the complete flow using the seeded data.

5. REQUIREMENTS

The following hardware and software requirements are sufficient for running the project in a normal college lab environment.

A. Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 / equivalent	Intel Core i5 or better
RAM	4 GB	8 GB or more
Storage	2 GB free space	5 GB free space or more
Display	Standard HD display	Full HD display
Network	Basic internet access for CDN-based chart loading	Stable internet connection

B. Software Requirements

Software	Purpose
Operating System	Windows, Linux, or macOS
VS Code	Source code editing and project management
Node.js	Backend runtime for server execution
MySQL / XAMPP	Database server and administration
Browser	Chrome, Edge, Firefox, or another modern browser
npm	Dependency installation and script running

Required Packages

- `express` for backend routing
- `mysql2` for database connectivity
- `dotenv` for environment configuration
- `bcrypt` for password hashing
- `jsonwebtoken` for token generation and verification
- `express-validator` for request validation
- `cors` for cross-origin support
- `nodemon` for development reloads

Environment File Example

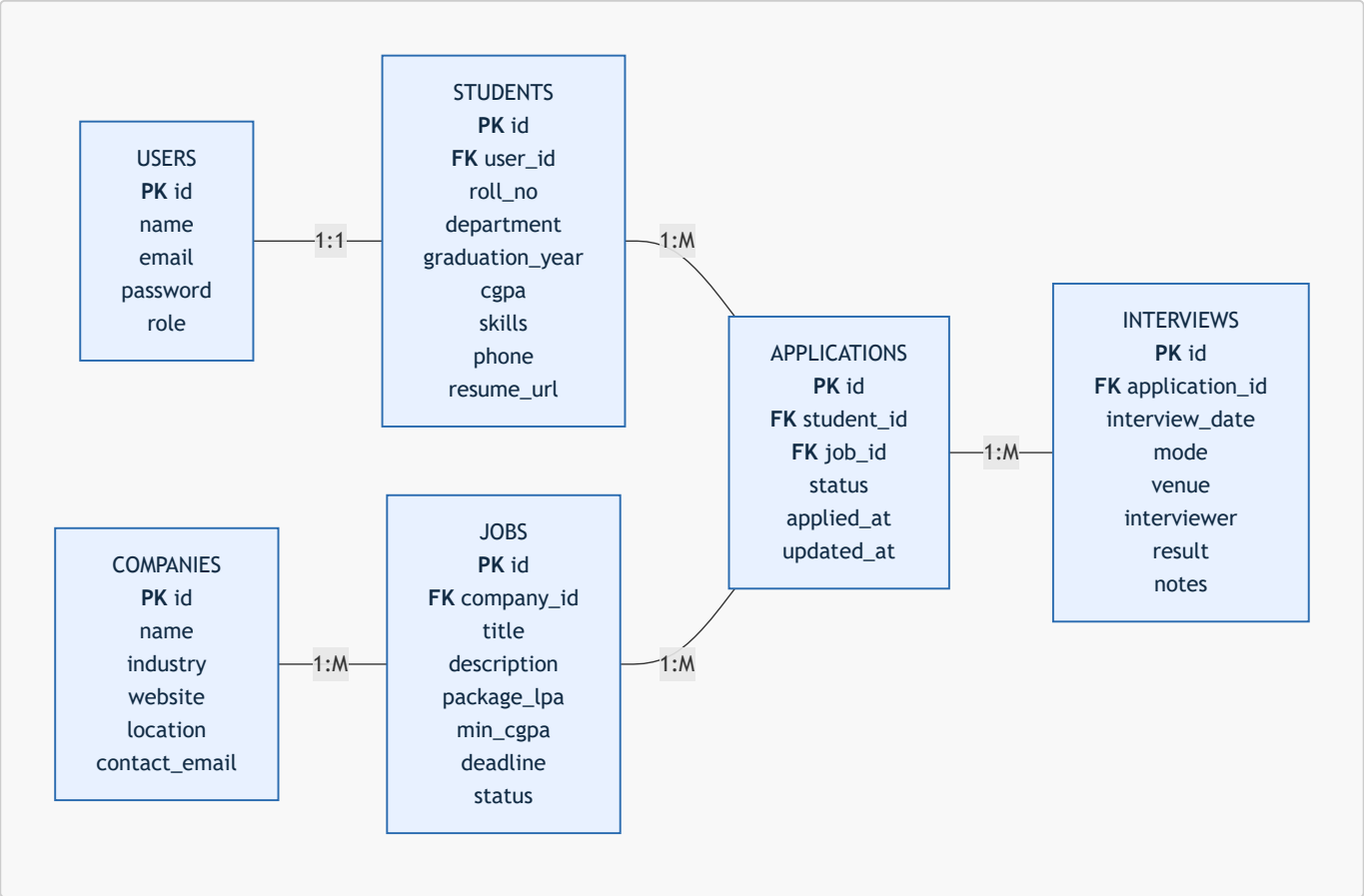
```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=placement_management
```

```
JWT_SECRET=mysecretkey
PORT=5000
```

6. E-R DIAGRAM

The ER model is built around six core entities and their placement workflow relationships. The design keeps authentication, student profile, recruiter data, job posting, application tracking, and interview scheduling in separate entities while linking them through foreign keys.

Clean ER Diagram



Textual Relationship View

```
[USERS]       1:1  [STUDENTS]
[COMPANIES]   1:M  [JOBS]
[STUDENTS]    M:N  [JOBS] via [APPLICATIONS]
[APPLICATIONS] 1:M  [INTERVIEWS]
```

Entity Notes

- **users** stores login identity for admin and student accounts.
- **students** stores academic and profile data linked to one user account.
- **companies** stores recruiter information used by job postings.
- **jobs** stores role, package, eligibility, location, and deadline.
- **applications** stores the link between a student and a job, along with the current stage.
- **interviews** stores the interview schedule and result attached to an application.

Keys and Constraints

- `users.id` is the primary key.
- `students.user_id` is a foreign key to `users.id` and is unique, enforcing one profile per login.
- `jobs.company_id` is a foreign key to `companies.id`.
- `applications.student_id` and `applications.job_id` are foreign keys and together are protected by a unique constraint to prevent duplicate applications.
- `interviews.application_id` is a foreign key to `applications.id`.

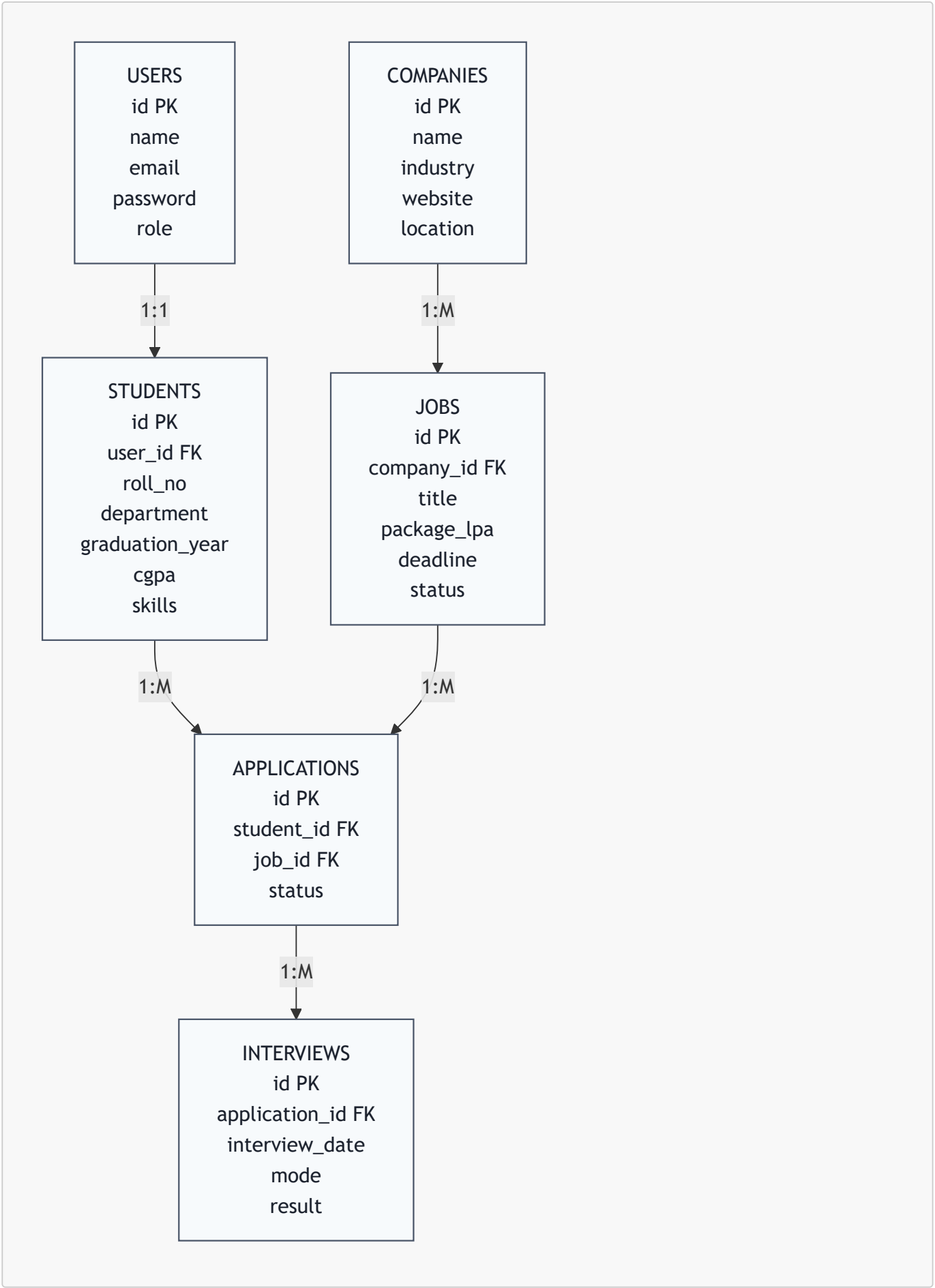
Why This ER Design Works

This structure prevents data duplication and keeps the system consistent. A student can apply to many jobs, but each application refers back to one student and one job. Likewise, interviews are not stored as a free-standing list; they remain tied to a specific application. This is the correct design for a DBMS-driven placement system because it supports accurate joins, analytics, and status tracking.

7. RELATIONAL DATABASE DESIGN

The relational design converts the ER model into six normalized tables. Each table stores one entity type and uses keys and constraints to preserve integrity. This design is suitable for a college placement workflow because the data must be joined frequently for dashboards, student tracking, interview schedules, and placement reports.

Relational Schema Diagram



Relationship Summary

- `users` is the parent table for student login records.

- `companies` is the parent table for jobs.
- `students` and `jobs` become connected through `applications`.
- `applications` is the bridge table that resolves the many-to-many relationship between students and jobs.
- `interviews` depends on applications and therefore inherits the placement context.

Table: users

Purpose: stores authentication identity for admins and students.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(120)	NOT NULL, UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Bcrypt hash
role	ENUM('admin','student')	NOT NULL	Access role
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation time

Table: students

Purpose: stores student academic and contact profile linked to users.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Student record ID
user_id	INT	FK -> users.id, UNIQUE, NOT NULL	Links to user login
roll_no	VARCHAR(40)	NOT NULL, UNIQUE	College roll number
department	VARCHAR(80)	NOT NULL	Department name
graduation_year	YEAR	NOT NULL	Expected graduation year
cgpa	DECIMAL(3,2)	CHECK 0 to 10	Academic score
skills	TEXT	NULL	Skill summary
phone	VARCHAR(20)	NULL	Contact number
resume_url	VARCHAR(255)	NULL	Resume link

Table: companies

Purpose: stores recruiter organization details.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Company ID
name	VARCHAR(120)	NOT NULL, UNIQUE	Company name
industry	VARCHAR(100)	NOT NULL	Business sector
website	VARCHAR(180)	NULL	Official URL

Column	Type	Key / Constraint	Purpose
location	VARCHAR(120)	NULL	City or branch location
contact_email	VARCHAR(120)	NULL	HR or campus contact
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

Table: jobs

Purpose: stores job openings posted by companies.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Job ID
company_id	INT	FK -> companies.id, NOT NULL	Posting company
title	VARCHAR(140)	NOT NULL	Job title
description	TEXT	NOT NULL	Role description
package_lpa	DECIMAL(5,2)	NOT NULL	Salary package in LPA
min_cgpa	DECIMAL(3,2)	DEFAULT 0	Minimum CGPA eligibility
location	VARCHAR(120)	NULL	Job location
deadline	DATE	NOT NULL	Application deadline
status	ENUM('open','closed')	DEFAULT 'open'	Open or closed posting
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Posting time

Table: applications

Purpose: stores student applications and status changes.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Application ID
student_id	INT	FK -> students.id, NOT NULL	Applicant student
job_id	INT	FK -> jobs.id, NOT NULL	Applied job
status	ENUM('applied','shortlisted','selected','rejected')	DEFAULT 'applied'	Application stage
applied_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Application time
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last status change
unique_application	UNIQUE(student_id, job_id)	Prevents duplicate application	One student per job

Table: interviews

Purpose: stores interview schedule and result information.

Column	Type	Key / Constraint	Purpose
id	INT	PK, AUTO_INCREMENT	Interview ID
application_id	INT	FK -> applications.id, NOT NULL	Related application
interview_date	DATETIME	NOT NULL	Date and time
mode	ENUM('online','offline')	NOT NULL	Interview mode
venue	VARCHAR(180)	NULL	Online link or physical venue
interviewer	VARCHAR(120)	NULL	Interviewer name
result	ENUM('scheduled','selected','rejected','on_hold')	DEFAULT 'scheduled'	Interview outcome
notes	TEXT	NULL	Additional remarks

SQL CREATE TABLE Examples

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'student') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL UNIQUE,
  roll_no VARCHAR(40) NOT NULL UNIQUE,
  department VARCHAR(80) NOT NULL,
  graduation_year YEAR NOT NULL,
  cgpa DECIMAL(3,2) NOT NULL CHECK (cgpa >= 0 AND cgpa <= 10),
  skills TEXT,
  phone VARCHAR(20),
  resume_url VARCHAR(255),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Relationship Notes

- `ON DELETE CASCADE` is used where dependent records should be removed automatically.
- The unique application constraint prevents duplicate applications from the same student for the same job.
- Foreign keys ensure that the database always references valid rows and does not contain broken links.

8. DATABASE NORMALIZATION

Normalization is the process of organizing data so that redundancy is reduced and update anomalies are avoided. The Placement Management System uses a normalized relational design because the workflow involves repeated references to students, jobs, companies, and interviews.

First Normal Form (1NF)

A table is in 1NF when every column contains atomic values and each record is uniquely identifiable. In this project, the `users`, `students`, `companies`, `jobs`, `applications`, and `interviews` tables all contain atomic fields such as name, email, department, and package. Repeating groups were avoided. For example, company information is not repeated inside every job row; instead, the job stores `company_id`.

Second Normal Form (2NF)

A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole primary key. The `applications` table demonstrates this idea clearly. Its primary key is `id`, while the pair `(student_id, job_id)` is also made unique to represent one application per student-job pair. Application status depends on the application record as a whole, not on either foreign key alone. Similarly, job title and package are stored in the jobs table and not inside applications, so data is not repeated.

Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and non-key attributes do not depend on other non-key attributes. This project follows 3NF by separating student identity from student profile, separating company details from job postings, and separating interview data from application records. For example, the company name is stored only in the `companies` table. Jobs reference the company through `company_id`. If the company name changes, only one row is updated.

How Normalization Is Applied in This Project

- Login data is kept in `users`.
- Student academic profile is kept in `students`.
- Recruiter information is kept in `companies`.
- Job postings are kept in `jobs`.
- Applications are kept in `applications`.
- Interviews are kept in `interviews`.

This design avoids storing the same student name, company name, or job title in multiple tables unnecessarily. It also makes the database easier to maintain.

Reduction of Redundancy

Redundancy is reduced in several ways:

- The student name is not stored repeatedly in applications; it is fetched through joins.
- Company details are not duplicated in jobs; the company ID is enough.
- Interview results are tied to the application, not recreated separately for the same student and job pair.

Example From the Project Tables

If Aarav Sharma applies for the Frontend Developer role at TechNova Solutions, the database stores:

- One user row for Aarav Sharma
- One student row linked to that user
- One company row for TechNova Solutions
- One job row for Frontend Developer
- One application row linking student and job
- One interview row if the application advances to interview

This is a compact and clean normalized structure.

9. DATA DICTIONARY

The data dictionary below describes the main tables, their attributes, and the role of each field in the system. It is useful during implementation, database maintenance, and viva explanation.

users

Table	Attribute	Data Type	Constraints	Description
users	id	INT	PK, AUTO_INCREMENT	Unique user ID
users	name	VARCHAR(100)	NOT NULL	Full name
users	email	VARCHAR(120)	NOT NULL, UNIQUE	Login email
users	password	VARCHAR(255)	NOT NULL	Hashed password
users	role	ENUM('admin','student')	NOT NULL	User role
users	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation time

students

Table	Attribute	Data Type	Constraints	Description
students	id	INT	PK, AUTO_INCREMENT	Student profile ID
students	user_id	INT	FK, UNIQUE, NOT NULL	Linked login user
students	roll_no	VARCHAR(40)	NOT NULL, UNIQUE	College roll number
students	department	VARCHAR(80)	NOT NULL	Department name
students	graduation_year	YEAR	NOT NULL	Expected graduation year
students	cgpa	DECIMAL(3,2)	CHECK 0 to 10	Academic score
students	skills	TEXT	NULL	Key skills
students	phone	VARCHAR(20)	NULL	Contact number
students	resume_url	VARCHAR(255)	NULL	Resume link

companies

Table	Attribute	Data Type	Constraints	Description
companies	id	INT	PK, AUTO_INCREMENT	Company ID
companies	name	VARCHAR(120)	NOT NULL, UNIQUE	Company name
companies	industry	VARCHAR(100)	NOT NULL	Sector
companies	website	VARCHAR(180)	NULL	Website URL
companies	location	VARCHAR(120)	NULL	City or office location
companies	contact_email	VARCHAR(120)	NULL	HR contact email

Table	Attribute	Data Type	Constraints	Description
companies	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

jobs

Table	Attribute	Data Type	Constraints	Description
jobs	id	INT	PK, AUTO_INCREMENT	Job ID
jobs	company_id	INT	FK, NOT NULL	Linked company
jobs	title	VARCHAR(140)	NOT NULL	Job title
jobs	description	TEXT	NOT NULL	Job details
jobs	package_lpa	DECIMAL(5,2)	NOT NULL	Salary package
jobs	min_cgpa	DECIMAL(3,2)	DEFAULT 0	Minimum CGPA
jobs	location	VARCHAR(120)	NULL	Job location
jobs	deadline	DATE	NOT NULL	Application deadline
jobs	status	ENUM('open','closed')	DEFAULT 'open'	Posting status
jobs	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Post creation time

applications

Table	Attribute	Data Type	Constraints	Description
applications	id	INT	PK, AUTO_INCREMENT	Application ID
applications	student_id	INT	FK, NOT NULL	Student reference
applications	job_id	INT	FK, NOT NULL	Job reference
applications	status	ENUM('applied','shortlisted','selected','rejected')	DEFAULT 'applied'	Current status
applications	applied_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Application timestamp
applications	updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last updated time

interviews

Table	Attribute	Data Type	Constraints	Description
interviews	id	INT	PK, AUTO_INCREMENT	Interview ID

Table	Attribute	Data Type	Constraints	Description
interviews	application_id	INT	FK, NOT NULL	Related application
interviews	interview_date	DATETIME	NOT NULL	Interview date and time
interviews	mode	ENUM('online','offline')	NOT NULL	Mode of interview
interviews	venue	VARCHAR(180)	NULL	Venue or online meeting link
interviews	interviewer	VARCHAR(120)	NULL	Interviewer name
interviews	result	ENUM('scheduled','selected','rejected','on_hold')	DEFAULT 'scheduled'	Interview outcome
interviews	notes	TEXT	NULL	Additional notes

Data Dictionary Observations

The dictionary shows that each table has a clear purpose and a small number of related fields. This separation makes the database easier to query and maintain. It also supports the dashboard queries, which rely on joins and grouped aggregations.

10. GRAPHICAL USER INTERFACE

The frontend of the Placement Management System is split into focused pages so that each user action has a clear screen. The interface is beginner-friendly, responsive, and easy to explain during viva.

Login Page

- Purpose: authenticate admin and student users.
- Fields: email, password, and role.
- Behavior: on success, the JWT token and user object are stored in `localStorage`.

Screenshot Placeholder: `[Login Page Screenshot Here]`

Dashboard

- Purpose: show placement summary and live status.
- Displays KPI cards for students, companies, jobs, applications, selections, rejections, and upcoming interviews.
- Includes recent applications, upcoming interviews, and top offers tables.
- Uses Chart.js graphs for quick visual analysis.

Screenshot Placeholder: `[Admin Dashboard Screenshot Here]`

Students Page

- Purpose: manage student profiles.
- Admin can create, update, view, and delete records.
- The form contains academic and contact fields such as roll number, department, CGPA, skills, phone, and resume URL.
- The table below the form lists all students with joined identity data.

Screenshot Placeholder: `[Students Page Screenshot Here]`

Companies Page

- Purpose: maintain recruiter information.
- Stores company name, industry, website, location, and contact email.
- Supports company-level records that later connect to job postings.

Screenshot Placeholder: `[Companies Page Screenshot Here]`

Jobs Page

- Purpose: publish and manage placement openings.
- Fields: title, description, package, minimum CGPA, location, deadline, and status.
- Students can view open postings and apply from the job cards.

Screenshot Placeholder: `[Jobs Page Screenshot Here]`

Applications Page

- Purpose: track student-job applications.
- Admin can update status to applied, shortlisted, selected, or rejected.

- Students can monitor the progress of their own applications.

Screenshot Placeholder: `[Applications Page Screenshot Here]`

Interviews Page

- Purpose: schedule interview rounds and store outcomes.
- Captures application reference, date and time, mode, venue, interviewer, result, and notes.
- Provides a clear history of interview events linked to applications.

Screenshot Placeholder: `[Interviews Page Screenshot Here]`

Analytics Dashboard

- Purpose: convert raw database data into visual insights.
- Charts included:
  - Students by department
  - Placement status
  - Company-wise hiring
  - Applications over time
  - Package distribution
  - Interview timeline
- Dashboard values update from the `/api/dashboard/analytics` endpoint.

Screenshot Placeholder: `[Analytics Dashboard Screenshot Here]`

Dark Mode

- Available across all major pages.
- The selected theme is stored in `localStorage`.
- Chart colors and labels are refreshed when the theme changes.
- Improves readability in low-light environments.

Navigation and Responsiveness

- Navigation is role-based.
- Admin users see dashboard, students, companies, jobs, applications, and interviews.
- Student users see dashboard, jobs, applications, and interviews.
- The layout adjusts for smaller screens through responsive cards, table wrappers, and form blocks.
- Shared JavaScript keeps page behavior consistent across the site.

11. SOURCE CODE

This project is organized into a clear backend and frontend structure. Each file has a focused responsibility, which makes the code easier to maintain and easier to explain in viva.

Important Backend Files

File	Role
`backend/server.js`	Starts the Express server and registers routes
`backend/src/config/db.js`	Creates the MySQL connection pool
`backend/src/controllers/`	Contains business logic
`backend/src/routes/`	Maps URLs to controllers
`backend/src/middleware/`	Handles auth, validation, and errors
`backend/src/utils/asyncHandler.js`	Wraps async controller functions

Important Frontend Files

File	Role
`frontend/index.html`	Login page
`frontend/admin-dashboard.html`	Admin analytics dashboard
`frontend/student-dashboard.html`	Student dashboard
`frontend/students.html`	Student management page
`frontend/companies.html`	Company management page
`frontend/jobs.html`	Job posting page
`frontend/applications.html`	Application tracking page
`frontend/interviews.html`	Interview scheduling page
`frontend/assets/js/api.js`	Shared fetch wrapper
`frontend/assets/js/auth.js`	Login state and role checks
`frontend/assets/js/ui.js`	Theme toggle, toasts, loader helpers
`frontend/assets/js/dashboard.js`	Dashboard and chart logic

Core Files

- `backend/server.js` starts Express, mounts APIs, and serves the frontend.
- `backend/src/config/db.js` creates the MySQL connection pool.
- `backend/src/controllers/*` contains the business logic for each module.
- `backend/src/routes/*` maps URLs to controller functions.
- `backend/src/middleware/*` handles authentication, validation, and errors.
- `frontend/assets/js/api.js` centralizes all `fetch()` requests.

- `frontend/assets/js/auth.js` manages token storage and role checks.
- `frontend/assets/js/ui.js` handles theme, loader, toast, and utility functions.
- `frontend/assets/js/dashboard.js` loads dashboard statistics and charts.

Important Code Snippets

Database Connection

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'placement_management',
  waitForConnections: true,
  connectionLimit: 10
});
```

This creates a reusable MySQL pool. It keeps the application responsive when multiple users send requests.

Login API

```
exports.login = asyncHandler(async (req, res) => {
  const { email, password, role } = req.body;
  const [rows] = await pool.query(
    'SELECT * FROM users WHERE email = ? AND role = ?',
    [email, role]
  );
  const user = rows[0];

  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ message: 'Invalid email, password, or role' });
  }

  res.json({ message: 'Login successful', token: createToken(user), user });
});
```

This code validates the credentials, compares the bcrypt hash, and returns a JWT token after a successful login.

JWT Middleware

```
function protect(req, res, next) {
  const authHeader = req.headers.authorization || '';
  const token = authHeader.startsWith('Bearer ') ? authHeader.split(' ')[1] : null;

  if (!token) return res.status(401).json({ message: 'Authentication token missing' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET || 'dev_secret');
    next();
```

```
  } catch {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
}
```

This middleware protects private routes and makes sure the request is coming from a logged-in user.

Analytics API

```
exports.getAnalytics = asyncHandler(async (req, res) => {
  const [studentsByDepartment] = await pool.query(`
    SELECT department AS label, COUNT(*) AS value
    FROM students
    GROUP BY department
    ORDER BY value DESC
  `);

  res.json({ studentsByDepartment });
});
```

This query returns chart-ready label and value pairs. The frontend passes them directly to Chart.js.

Shared Fetch Wrapper

```
async function apiRequest(endpoint, options = {}) {
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  const token = localStorage.getItem('token');
  if (token) headers.Authorization = `Bearer ${token}`;

  const response = await fetch(`/api${endpoint}`, { ...options, headers });
  const data = await response.json();
  if (!response.ok) throw new Error(data.message || 'Request failed');
  return data;
}
```

This wrapper removes repeated request code and automatically attaches the JWT token when available.

12. CONCLUSION

The Placement Management System successfully demonstrates how a DBMS project can be turned into a real working web application. It supports secure login, structured data storage, student management, company management, job posting, application tracking, interview scheduling, and dashboard analytics. By combining a normalized MySQL schema with a Node.js and Express.js backend, the project provides a reliable and maintainable solution for campus placement operations.

One major achievement of the project is the integration of multiple modules into a single workflow. The system allows the placement cell to keep records in a consistent manner and gives students a simple way to view jobs and track their applications. The analytics dashboard adds value by presenting placement data visually, which helps during reporting, planning, and review meetings.

The learning outcomes from this project are significant. The project improved understanding of relational database design, primary and foreign keys, normalization, REST API development, JWT-based authentication, bcrypt password hashing, frontend-backend communication, and chart-based reporting. It also showed how modular code can improve maintainability and how the same backend can support both admin and student roles.

The advantages of the system include centralized control, reduced manual work, better data accuracy, faster updates, improved transparency, and easier reporting. The role-based structure ensures that sensitive operations remain protected. The use of JavaScript on both the frontend and backend also keeps the development process consistent and beginner-friendly.

Future improvements can make the project even stronger. Useful additions include company login, email alerts, advanced search filters, resume parsing, automatic eligibility checking, export to PDF or Excel, attendance tracking for interview rounds, and AI-based placement insights. These enhancements would turn the project into a more advanced campus recruitment platform.

In conclusion, the Placement Management System is a complete and practical DBMS project that satisfies academic requirements while also reflecting how real placement software works. It is technically correct, structured, and easy to explain during viva. The project shows the value of combining database design, secure backend development, and interactive frontend design in one well-organized system.

References

1. Node.js Official Documentation - https://nodejs.org/en/docs
2. Express.js Official Guide - https://expressjs.com/
3. MySQL Reference Manual - https://dev.mysql.com/doc/
4. JWT Introduction - https://jwt.io/introduction
5. bcrypt Package Documentation - https://www.npmjs.com/package/bcrypt
6. Chart.js Documentation - https://www.chartjs.org/docs/latest/
7. MDN Web Docs for HTML, CSS, and JavaScript - https://developer.mozilla.org/
8. Project source files in this repository: `backend/server.js`, `backend/src/controllers/*`, `backend/src/routes/*`, `backend/src/middleware/*`, `frontend/assets/js/*`, and `database.sql`

Bibliography

- Silberschatz, A., Korth, H. F., and Sudarshan, S., *Database System Concepts*.
- Elmasri, R., and Navathe, S. B., *Fundamentals of Database Systems*.
- W3C and MDN resources for web standards and page structure.
- Official documentation for Node.js, Express.js, MySQL, JWT, bcrypt, and Chart.js.
- The actual Placement Management System project files present in the folder.